

House Price Estimation Using Gradient Boosting

Wyatt Flatt

4/20/26

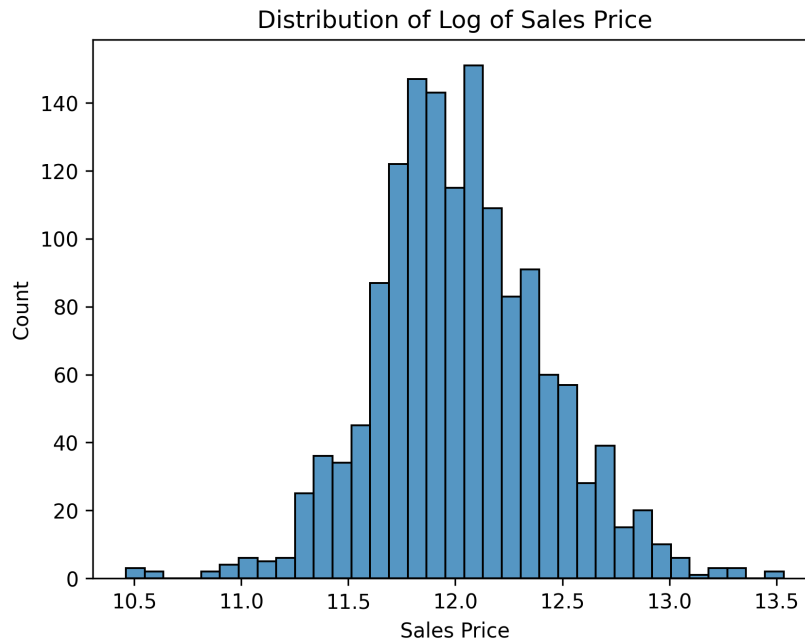
I. Introduction and Exploratory Analysis

The Ames housing dataset is a collection of observations on houses from Ames, Iowa. This data is used to test advanced machine-learning algorithms' ability to predict the sales prices of houses using a collection of explanatory features by allowing individuals to submit their models' predictions and receive a score. The explanatory features associated with each observation include a house's physical characteristics, room information, location in relation to other objects, and condition. This collection of explanatory features includes both quantitative and qualitative information. The observations are already separated into train and test sets, leaving the focus of this project to be maximizing predictive accuracy on the target feature, sales price, using a gradient boosting model.

The structure of the data can be explained in a few pieces of summary information. Within the training data, there are 79 explanatory features for 1,460 observations/houses. The feature set is made up of 36 numeric and 43 categorical features. Among the feature set, 19 features contain NA values, with the number of NA values ranging from 1 to 1,453 within the features.

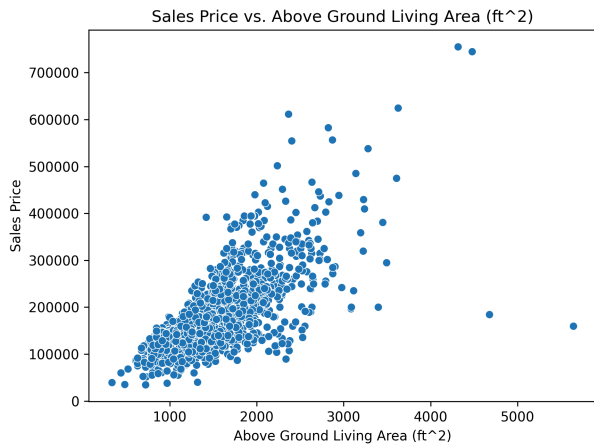
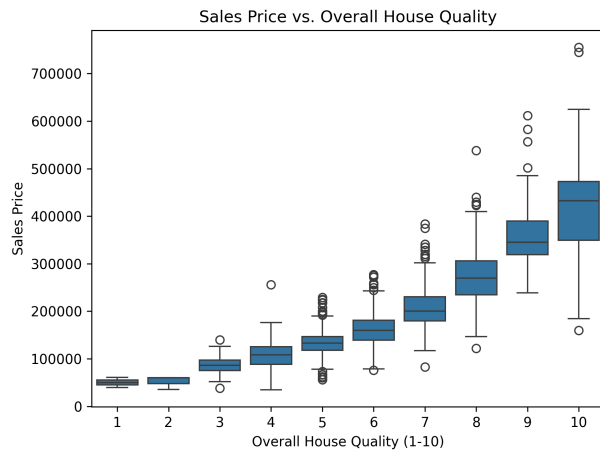
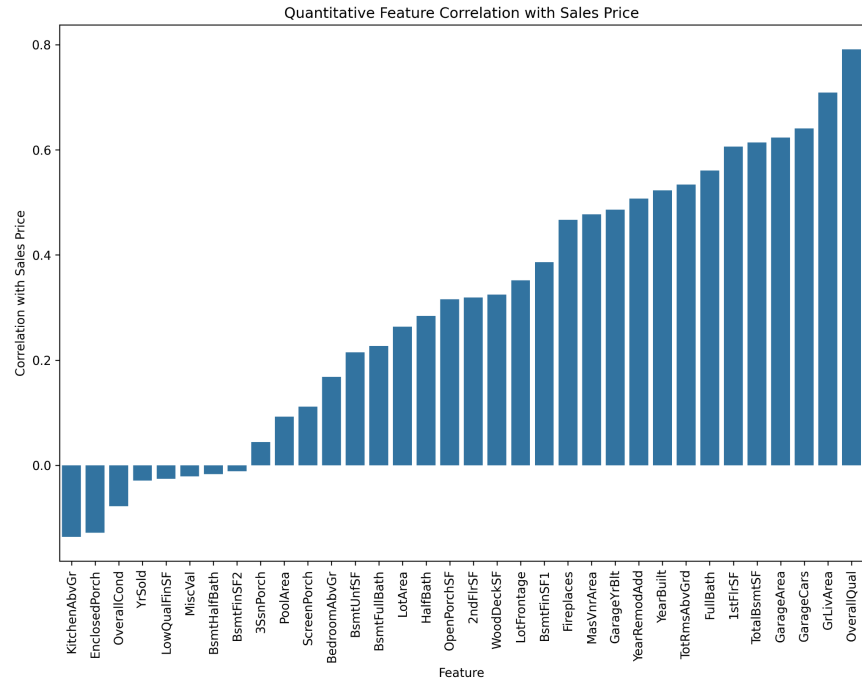
In the early stages of exploratory analysis, it was determined that the distribution of sales prices in the training data was right-skewed with a small number of extreme outliers. Houses most frequently fell within the \$100,000 to \$200,000 range. By applying a natural log transformation to the sale prices, the distribution became relatively normal (Graph 1); however, this transformation was not used in the final model creation.

Graph 1



To better understand the relationships between features and sales price, features were separated into quantitative and categorical groups. Analyses were then performed differently based on which group the feature belonged to. For quantitative features, their relationships with sales price were first explored via the correlation between each feature and sales price (Graph 2). The resulting correlations demonstrated that features related to the size and quality of a house and its garage had large positive associations with its sales price. Most notably, the overall quality of the house on a scale from 1 to 10 (Graph 3) and the above-ground living area measured in square feet (Graph 4) were most positively related to the sales price, with correlations of 0.79 and 0.71, respectively. Another notable finding from the correlations was that the overall condition of a house on a scale from 1 to 10 was not positively related to the sales price. It was also discovered that many of the features that were highly correlated with sales price were highly correlated with one another.

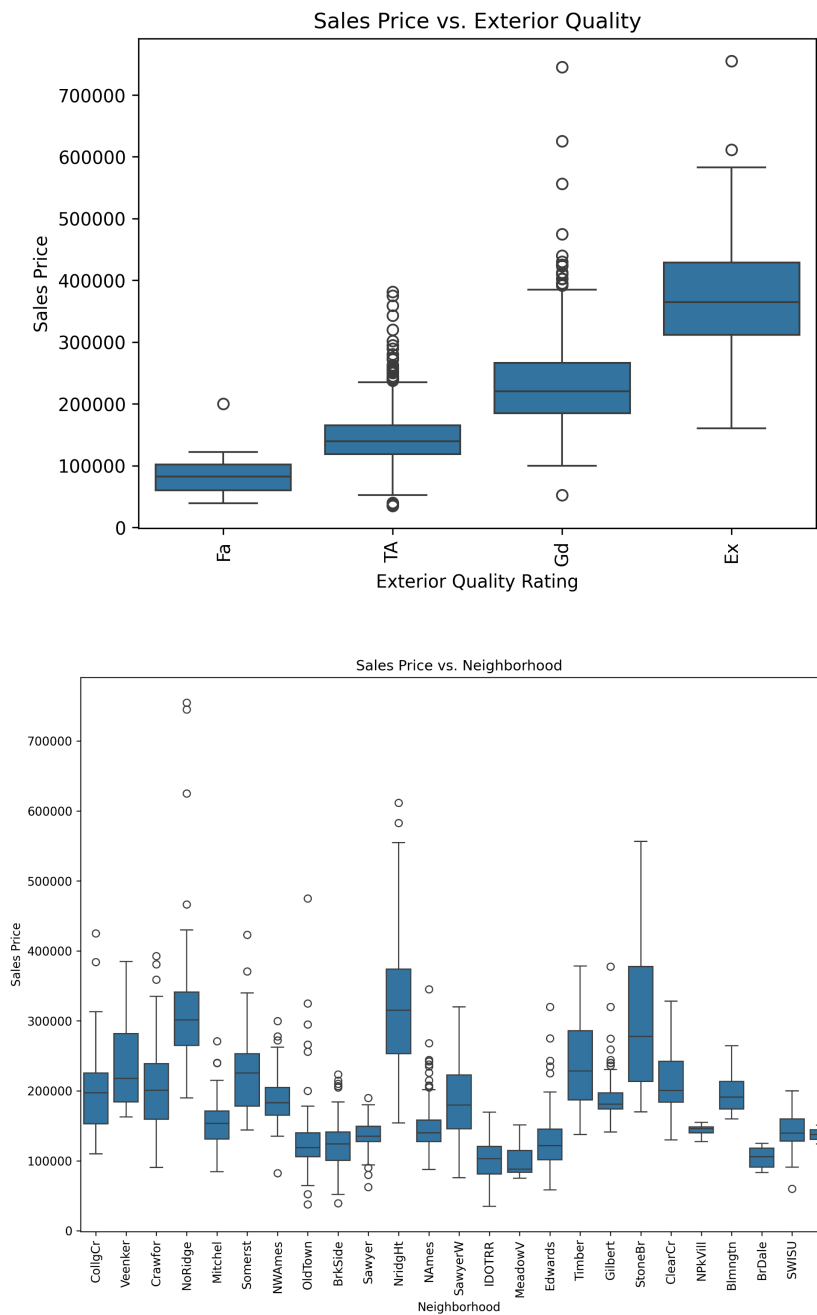
Graphs 2-4



Among categorical features, boxplots were used to determine how the average sales price differed between the category levels of each feature. The features that appeared to have some of the largest sales price differences between categories were the exterior material quality of a house and the neighborhood in which a house resides. There is a clear increase in the average sales price based on the quality of the exterior material on a scale of fair to excellent (Graph 5). Regarding neighborhoods, houses residing in the Northridge, Northridge Heights, and Stone

Brook regions of Ames tend to have higher sales prices on average than those in other regions (Graph 6).

Graphs 5-6



In addition to graphical analysis, differences in sale prices were examined based on the presence or absence of specific features. Comparing the average sale prices of houses with each

feature to those without showed that alley access, a masonry veneer, and fencing were associated with tens of thousands of dollars in price differences.

II. Preprocessing

Preprocessing was separated into three distinct sections: feature engineering/creation, feature removal, and pipeline imputation and feature encoding. For the first two sections, the methods were applied to both the training and testing data to ensure similar features and feature formats across the entirety of the data.

In section one, feature creation was used to capture differences between houses with and without alley access, a masonry veneer, and fencing while reducing the dimensionality of the data. For each of these original features, a new binary variable was created to indicate the presence or absence of the characteristic described in the original feature. By using these binary indicators and removing the original alley, masonry veneer type, and fence features, the model retained information about whether these features exist while reducing the number of categories and encoded features during model creation, reducing computational expense.

To further reduce dimensionality and improve modeling performance, certain ordinal categorical features were transformed into numeric features with equal spacing between categories. These features included the quality of a house's exterior material, basement, garage, fireplace, kitchen, and heating, as well as the condition of a house's exterior material, basement, and garage. Each of these features had an ordinal rating system comprised of the following labels: poor, fair, typical/average, good, and excellent. Because these ordinal categories can also be represented using a numeric rating system, they were changed as such. By converting these features from categorical to numeric, this ensured that the ordinal relationship of the categories

was maintained during the underlying decision tree splits of the gradient boosting model, instead of being removed during one-hot encoding. In addition, the reduced number of encoded categories that resulted from this alteration reduced the sparsity of the data.

For many categorical features in the data, NA values were used to represent the lack of a feature rather than missing information. Because the lack of a feature can be highly predictive, multiple features' NA values were imputed with a new category labeled as “None.” This method was applied to the following features: the exposure of the basement, the rating of the basement’s finished area, a second rating of the basement’s finished area (if there were multiple types), the garage type, and the interior finish of the garage.

In section two of preprocessing, features were dropped from the data. In total, only five features were dropped. The alley, masonry veneer type, and fence features were dropped since they were already reduced into more compact features in section one of preprocessing. The other features that were dropped included a categorical feature containing information about miscellaneous housing items that were not covered by other features, and a feature indicating the quality of the house’s pool. Because these features had very few observations split into multiple categories, they were too sparse and likely not informative for modeling purposes.

Section three of preprocessing involved the imputation of missing features and the encoding of categorical features. Because different preprocessing steps were required for different features within the data set, this stage of preprocessing was implemented via pipelines. Pipelines are used to string together a series of transformations and model-fitting steps on the data being piped in (Scikit-learn, n.d.-c). This allowed for a reproducible combination of preprocessing and model-creation steps using Scikit-learn.

The primary reason for using pipelines to model in this way was to ensure that preprocessing steps were correctly applied within each training fold during k-fold cross-validation, preventing data leakage. If imputation steps were handled before cross-validation, each model created in each iteration of cross-validation would learn from the imputation, giving an underestimated test error and potentially resulting an incorrect optimal model being chosen. By creating a pipeline, imputation was handled separately in each fold, preventing data leakage.

First, a pipeline was created to handle the preprocessing steps for numeric features. This pipeline was used to impute the missing values within each numeric feature with the median value of that feature. Another pipeline was created to handle the preprocessing steps for categorical features. This pipeline contains two steps, which are as follows: imputing the missing values within each categorical feature with the most frequently occurring category within that feature and one-hot encoding categorical features to convert each separate category of each feature into a binary variable with a value of 1 or 0 to indicate the presence or absence of that category. This encoding is necessary for many models to interpret the presence of a category (Geeks for Geeks, 2026). Encoding was handled such that if a category of a feature were observed in the testing data but not in the training data, the test value would simply be encoded as a 0 for all categories encoded during the fitting process, essentially ignoring the testing feature's category.

Lastly, a final preprocessing pipeline was created by combining the preprocessing pipelines for both numeric and categorical features using Scikit-learn's ColumnTransformer function, which enabled different transformations to be applied to specified subsets of features

within the dataset (Scikit-learn, n.d.-a). This ensured that numeric and categorical preprocessing steps were applied appropriately.

III. Modeling

The primary modeling framework used for sales price estimation was gradient boosting (GB) regression. This is an ensemble method that combines many weak decision tree regressors, meaning they have few splits, sequentially by fitting new trees to the residuals of the previous tree. The estimated residuals for observations are scaled via a learning rate to avoid over- or under-correcting predictions when added to the existing ensemble. This process is repeated over a fixed number of trees. The GB framework was chosen over other frameworks, such as random forest (RF), because I wanted to use an ensemble method, and GB models often outperform RF models if overfitting is handled (Geeks for Geeks, 2024).

Within this framework, both standard gradient boosting (GB) and stochastic gradient boosting (SGB) variants were considered. The stochastic version attempts to reduce overfitting by introducing increased randomness during the underlying decision tree creation process. This is achieved by fitting each tree using a subsample of the training data, which decorrelates the sequential trees and reduces overfitting (Scikit-learn, n.d.-b).

The model selection process was performed using five-fold cross-validation over a set grid of hyperparameters. The final model was refit using the combination of hyperparameters that minimized the cross-validated root mean squared error (RMSE). Because our training data contained 1,460 observations, five-fold cross-validation was chosen so that the number of observations was divisible by the number of folds, resulting in 292 observations per fold. Additionally, seeds were set for the gradient boosting model object and cross-validation indices

to ensure reproducibility of cross-validation results. RMSE was used as the model selection metric due to the competition scores being evaluated based on the RMSE between the log of predicted sales prices and the log of observed sales prices.

The hyperparameter grid used in the cross-validation process included values for the number of trees, learning rate, maximum depth of individual trees, minimum number of samples required in a leaf node, and the subsample proportion of the training data used for each tree. The number of trees controls the number of decision trees used in sequence during model creation. If the number of trees is too small, the sequenced trees may not provide enough correction for the residual errors by the end of the sequence. Conversely, if the number is too large, the sequence may overfit from corrections of the sequenced trees by fitting to noise.

The learning rate is used in conjunction with the number of trees. Each tree's predictions are scaled using the learning rate before being added to the overall model to avoid over- or under-emphasizing the fix from each tree. If the learning rate is too large, corrections from sequential trees may overshoot the actual correction that was needed. If the learning rate is too low, the overall model may take enormous amounts of time and trees to correct the sequential predictions.

The maximum depth of individual trees controls the complexity of the underlying decision trees. A large number of splits allows the trees to capture more detailed patterns, reducing bias. In contrast, fewer splits account for only the most influential splits based on the features, making the trees more generalized with a lower variance. The minimum number of samples required in a leaf node serves a similar purpose. The fewer observations required in nodes, the more the underlying decision trees can split to capture detailed effects, reducing bias.

With a larger requirement, the remaining nodes average many values to achieve the nodes' predictions, reducing a tree's variance.

The proportion of the training data considered during tree creation indicates whether the model is a GB or SGB model. If the proportion is anything less than 1, the underlying trees are created on a subset of the training data, meaning the final model is an SGB model (Scikit-learn, n.d.-b). As the subset gets smaller, trees are fit using fewer of the same observations, further decorrelating sequential trees. This helps control the variance of the gradient boosting model.

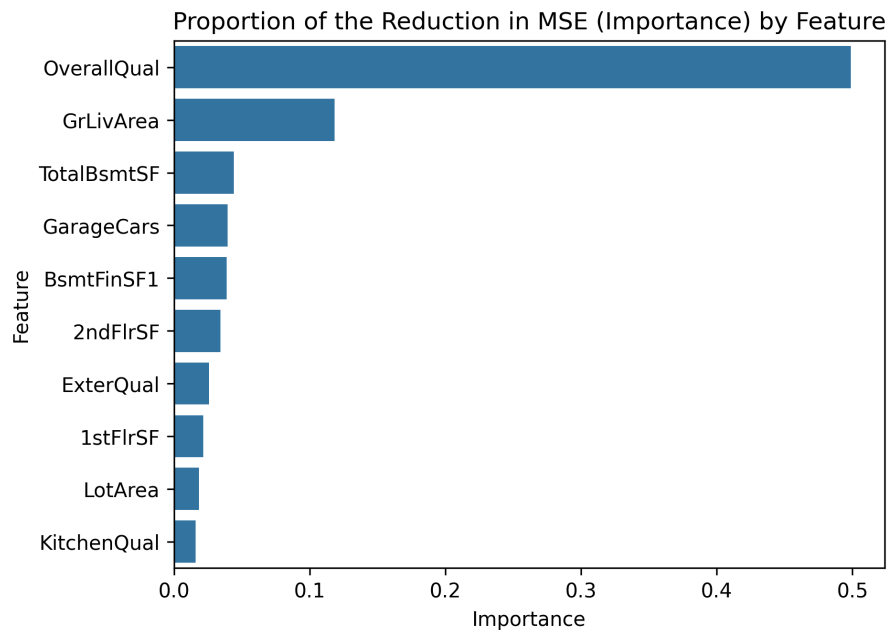
IV. Model Results

Based on the cross-validation procedure, the RMSE-minimizing model was a stochastic gradient boosting model. This model was fit using 1500 trees, a learning rate of .01, a maximum depth of 4 splits, a minimum number of samples required in a leaf node of 1, and 80 percent of the training observations to create individual decision trees. With this SGB model, the sales prices for the test data were predicted and submitted on the Kaggle competition page, resulting in an RMSE of .12859 between the log of the SGB model's sales price predictions and the log of the observed sales prices.

Feature importances, the proportion of the total decrease in the mean squared error (MSE) from splitting on a given feature across all trees, were used to determine the relative importance of features on predictive ability. The top ten features are presented in Graph 7. The overall quality of a house was determined to be the most important feature, accounting for 49.88 percent of the reduction in the MSE across the entire SGB model. The above-ground living area feature was also largely important, accounting for 11.84 percent of the reduction. These features were expected to be among the most important given their exceedingly large positive correlation

with sales price, but the reductions attributed to these features are likely overestimated due to the high correlation between features.

Graph 7



V. Limitations and Improvements

Ultimately, the SGB model achieved a respectable RMSE compared to other models in the Kaggle competition, but improvements could be made to the speed of the model. The code used to perform the cross-validation model selection took two hours to run. A long runtime is expected due to GB models creating trees sequentially rather than in parallel, like random forest or similar algorithms. Because tree creation is sequential and residuals need to be calculated continuously throughout a GB algorithm, the model does not scale well with the number of observations. Larger volumes of data would cause even longer runtimes as more residuals are calculated for the same number of trees.

One approach that could help reduce the algorithm's runtime would be to use a single value for the minimum number of samples required in a leaf node in the parameter grid that was used during cross-validation. Because max depth and the minimum number of samples required in a leaf node are both used to control the complexity of the sequentially created trees, the model would likely not suffer much in predictive ability if a single, reasonable value was used for the leaf node requirement in all combinations of hyperparameters. This would likely not affect predictive ability too harshly, but it would reduce the number of hyperparameter combinations attempted, reducing the number of times the more computationally expensive combinations are run. In turn, this would reduce the algorithm's runtime.

Another approach would be to use an XGBoost framework instead of a gradient boosting framework. XGBoost is built to optimize the speed of sequential tree creation, thereby significantly reducing the runtime of the cross-validation process. By using an XGBoost model, the cross-validation hyperparameter grid could be further expanded to explore new combinations. This could result in a more accurate model. Since XGBoost could help achieve both a faster runtime and a more accurate model, it is a natural next step to consider.

References

Geeks for Geeks. (April 9, 2024). *Gradient Boosting vs Random Forest*.

<https://www.geeksforgeeks.org/machine-learning/gradient-boosting-vs-random-forest/>

Geeks for Geeks. (March 12, 2026). *One Hot Encoding in Machine Learning*.

<https://www.geeksforgeeks.org/machine-learning/ml-one-hot-encoding/>

Scikit-learn. (n.d.-a). *ColumnTransformer*.

<https://scikit-learn.org/stable/modules/generated/sklearn.compose.ColumnTransformer.html>

Scikit-learn. (n.d.-b). *GradientBoostingRegressor*.

<https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.GradientBoostingRegressor.html>

Scikit-learn. (n.d.-c). *Pipeline*.

<https://scikit-learn.org/stable/modules/generated/sklearn.pipeline.Pipeline.html>